

Programming Language Concepts Continuations

Peter Sestoft
Niels Hallenberg
2026-07-24



Today

- Continuations
 - Tail-calls and accumulating parameters
 - Continuations and continuation-passing style
 - Continuation-based interpreters and exceptions
 - Continuations and backtracking

Compiling with no Tail Calls

```
int id(int b) {  
    return b;  
}
```

```
int f(int a) {  
    int b;  
    b = a + 1;  
    return id(b);  
}
```

```
void main() {  
    print f(2);  
}
```

Stack frame for **f**
remains on the stack
while **id** is called

```
[ 10: LDRCS]  
[ 4 -999 ]{47: CS 0}  
[ 4 -999 2 ]{49: CALL 5}  
[ 4 -999 52 2 2 ]{16: INCSP 1}  
[ 4 -999 52 2 2 0 ]{18: GETBP}  
[ 4 -999 52 2 2 0 4 ]{19: CSTI 1}  
[ 4 -999 52 2 2 0 4 1 ]{21: ADD}  
[ 4 -999 52 2 2 0 5 ]{22: GETBP}  
[ 4 -999 52 2 2 0 5 4 ]{23: CSTI 0}  
[ 4 -999 52 2 2 0 5 4 0 ]{25: ADD}  
[ 4 -999 52 2 2 0 5 4 ]{26: LDI}  
[ 4 -999 52 2 2 0 5 2 ]{27: CSTI 1}  
[ 4 -999 52 2 2 0 5 2 1 ]{29: ADD}  
[ 4 -999 52 2 2 0 5 3 ]{30: STI}  
[ 4 -999 52 2 2 3 3 ]{31: INCSP -1}  
[ 4 -999 52 2 2 3 ]{33: GETBP}  
[ 4 -999 52 2 2 3 4 ]{34: CSTI 1}  
[ 4 -999 52 2 2 3 4 1 ]{36: ADD}  
[ 4 -999 52 2 2 3 5 ]{37: LDI}  
[ 4 -999 52 2 2 3 3 ]{38: CALL id}  
[ 4 -999 52 2 2 3 41 4 3 ]{5: GETBP}  
[ 4 -999 52 2 2 3 41 4 3 8 ]{6: CSTI 0}  
[ 4 -999 52 2 2 3 41 4 3 8 0 ]{8: ADD}  
[ 4 -999 52 2 2 3 41 4 3 8 ]{9: LDI}  
[ 4 -999 52 2 2 3 41 4 3 3 ]{10: RET 1}  
[ 4 -999 52 2 2 3 3 ]{41: RET 2}  
[ 4 -999 3 ]{52: PRINTI}  
3 [ 4 -999 3 ]{53: INCSP -1}  
[ 4 -999 ]{55: INCSP 0}  
[ 4 -999 ]{57: RET -1}  
[ -999 ]{4: STOP}
```

Compiling with Tail Calls

```
int id(int b) {
    return b;
}

int f(int a) {
    int b;
    b = a + 1;
    return id(b);
}

void main() {
    print f(2);
}
```

Stack frame for **f**
is over-written
by frame for **id**.

```
[ 1/0: LDARGS}
main 0 32}
[ 4 -999 ]{32: CSTI 2}
[ 4 -999 2 ]{34: f 9}
[ 4 -999 37 2 2 ]{9: INCSP 1}
[ 4 -999 37 2 2 0 ]{11: GETBP}
[ 4 -999 37 2 2 0 4 ]{12: CSTI 1}
[ 4 -999 37 2 2 0 4 1 ]{14: ADD}
[ 4 -999 37 2 2 0 5 ]{15: GETBP}
[ 4 -999 37 2 2 0 5 4 ]{16: LDI}
[ 4 -999 37 2 2 0 5 2 ]{17: CSTI 1}
[ 4 -999 37 2 2 0 5 2 1 ]{19: ADD}
[ 4 -999 37 2 2 0 5 3 ]{20: STI}
[ 4 -999 37 2 2 3 3 ]{21: INCSP -1}
[ 4 -999 37 2 2 3 ]{23: GETBP}
[ 4 -999 37 2 2 3 4 ]{24: CSTI 1}
[ 4 -999 37 2 2 3 4 1 ]{26: ADD}
[ 4 -999 37 2 2 3 5 1 ]{27: LDI}
[ 4 -999 37 2 2 3 5 1 2 5 ]{28: TCALL 1 2 5}
[ 4 -999 37 2 3 ]{5: GETBP}
[ 4 -999 37 2 3 4 ]{6: LDI}
[ 4 -999 37 2 3 3 ]{7: RET 1}
[ 4 -999 3 ]{37: PRINTI}
[ 4 -999 3 ]{38: RET 0}
[ 3 ]{4: STOP}
```

Tail calls matter

Compiling for non-tail calls

- Create stack frame when calling a function.
- Remove stack frame when returning from a function.

Compiling for tail calls

- Stack frame for calling function is overwritten with stack frame for called function.
- Return address and old base pointer is kept from calling functions stack frame.

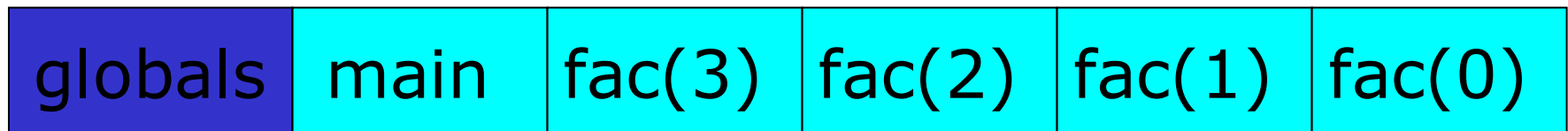
Recursive functions

Stack represents "continuation"

```
let rec facr n =  
  if n=0 then 1  
  else n * facr(n-1)
```

```
facr 3  
==> 3 * facr 2  
==> 3 * (2 * facr 1)  
==> 3 * (2 * (1 * facr 0))  
==> 3 * (2 * (1 * 1))  
==> 3 * (2 * 1)  
==> 3 * 2  
==> 6
```

- One stack frame per recursive call
- Each stack frame represents “remember to *multiply result by n*”
- Together the stack frames represent the current *continuation*: “what must be done to the result of this expression”



3* (

2* (

1* (



Tail-recursive functions, iteration

- Rewrite facr with accumulating parameter r

```
let rec faca n a =  
  if n=0 then a  
  else faca (n-1) (n * a)
```

$\text{faca } n \ a = a * (\text{facr } n)$

$\text{facr } n = \text{faca } n \ 1$

```
faca 3 1  
==> faca 2 3  
==> faca 1 6  
==> faca 0 6  
==> 6
```

Stack space used is independent of argument n!

Which Calls Are Tail Calls?

Assuming e is in tail position, then

Expression e

let $x = e_r$ in e_b end

$e_1 + e_2$

if e_1 then e_2 else e_3

let $f x = e_r$ in e_b end

$f e$

Status of sub-expressions

e_b is in tail position, e_r is not

neither e_1 nor e_2 is in tail position

e_2 and e_3 are in tail position, e_1 is not

e_b is in tail position, e_r is not

e is not in tail position

Below, g is in tail position and h is not.

$g\ 1$

$g(h\ 1)$

$h\ 1 + h\ 2$

if $1=2$ then $g\ 3$ else $g(h\ 4)$

let $x = h\ 1$ in $g\ x$ end

let $x = h\ 1$ in if $x=2$ then $g\ x$ else $g\ 3$ end

let $x = h\ 1$ in $g(\text{if } x=2 \text{ then } h\ x \text{ else } h\ 3)$ end

let $x = h\ 1$ in let $y = h\ 2$ in $g(x + y)$ end end



Making tail calls with Continuations

- Continuation = “the rest of a computation”
- Continuation-passing style = CPS:
 - Every function has a continuation argument **k**
 - Do not return **res**; instead call **k(res)**
 - The continuation “knows what to do with **res**”

Normal

```
let rec facr n =  
  if n=0 then 1  
  else n * facr(n-1)
```

Continuation
parameter

Don't return,
instead call k

New
continuation

CPS

```
let rec facc n k =  
  if n=0 then k 1  
  else facc (n-1) (fun v -> k(n * v))
```



Uses of continuations

- A function in CPS can sometimes be rewritten to use an accumulating parameter, saving memory
- A function in CPS can sometimes stop the computation early, saving time
- An interpreter in CPS can model exceptions and exception handling such as **try-catch**
- Continuations can implement expressions with multiple results, as in Icon and Prolog
- Continuation-thinking helps on-the-fly optimization in the micro-C compiler (next week's lecture)
- Continuations can be used to structure web dialogs
- Continuations have many other more magical uses

CPS = continuation-passing style



Deriving a CPS version of facr

```
let rec facr n =  
  if n=0 then 1  
  else n * facr(n-1)
```

```
let id = fun v -> v
```

```
let rec facc n k =  
  if n=0 then ...  
  else ...
```

```
facc n k = k(facr n)
```

```
facr n = facc n id
```

```
let rec facc n k =  
  if n=0 then k 1  
  else ...
```

```
let rec facc n k =  
  if n=0 then k 1  
  else facc (n-1) <new continuation>
```

```
let rec facc n k =  
  if n=0 then k 1  
  else facc (n-1) (fun v -> k(n * v))
```



Evaluating facc n id

```
let rec facc n k =  
    if n=0 then k 1  
    else facc (n-1) (fun v -> k(n * v))
```

```
let id = fun v -> v
```

```
facc 3 id  
==> facc 2 (fun v -> id(3 * v))  
==> facc 1 (fun w -> (fun v -> id(3 * v)) (2 * w))  
==> facc 0 (fun u -> (fun w -> (fun v -> id(3 * v)) (2 * w)) (1 * u))  
==> (fun u -> (fun w -> (fun v -> id(3 * v)) (2 * w)) (1 * u)) 1  
==> (fun w -> (fun v -> id(3 * v)) (2 * w)) (1 * 1)  
==> (fun w -> (fun v -> id(3 * v)) (2 * w)) 1  
==> (fun v -> id(3 * v)) (2 * 1)  
==> (fun v -> id(3 * v)) 2  
==> id(3 * 2)  
==> id 6  
==> 6
```

Uses no stack space!

Uses heap for
continuations



Joint exercise

- Given this function

```
let rec prod xs =  
  match xs with  
  | []      -> 1  
  | x::xr   -> x * prod xr
```

- The accumulating parameter version:

```
let rec proda xs a =  
  match xs with  
  | []      -> a  
  | x::xr   -> proda xr (x*a)
```

Joint exercise

- Given this function

```
let rec prod xs =  
  match xs with  
  | []      -> 1  
  | x::xr   -> x * prod xr
```

- Find the continuation-passing version, form:

```
let rec prodc xs k =  
  match xs with  
  | []      -> ...  
  | x::xr   -> ...
```

Deriving a CPS version of prod

```
let rec prod xs =  
  match xs with  
  | []      -> 1  
  | x::xr   -> x * prod xr
```

```
let id = fun v -> v
```

```
let rec prodc xs k =  
  match xs with  
  [] -> ...  
  | x::xr -> ...
```

```
prodc xs k = k(prod xs)
```

```
let rec prodc xs k =  
  match xs with  
  [] -> k 1  
  | x::xr -> ...
```

```
prod xs = prodc xs id
```

```
let rec prodc xs k =  
  match xs with  
  [] -> k 1  
  | x::xr -> prodc xr <new continuation>
```

```
let rec prodc xs k =  
  match xs with  
  [] -> k 1  
  | x::xr -> prodc xr (fun v -> k(v*x))
```

Evaluating prodc n id

```
let rec prodc xs k =  
  match xs with  
    [] -> k 1  
  | x::xr -> prodc xr (fun v -> k(v*x))  
  
let id = fun v -> v
```

```
prodc [1;2;3] id  
==> prodc [2;3] (fun v -> id(v*1))  
==> prodc [3] (fun w -> (fun v -> id(v*1)) (w*2))  
==> prodc [] (fun u -> (fun w -> (fun v -> id(v*1)) (w*2)) (u*3))  
==> (fun u -> (fun w -> (fun v -> id(v*1)) (w*2)) (u*3)) 1  
==> (fun w -> (fun v -> id(v*1)) (w*2)) (1*3)  
==> (fun w -> (fun v -> id(v*1)) (w*2)) 3  
==> (fun v -> id(v*1)) (3*2)  
==> (fun v -> id(v*1)) 6  
==> id(6*1)  
==> id 6  
==> 6
```

Uses no stack space!

Uses heap for
continuations

Continuations and accumulating parameters

- Both `(facc n k)` and `(faci n r)` are tail-recursive
- What relation between `k` and `r`?
- In fact, `k` always has form `fun u -> r*u`
 - To begin with, `k = (fun u -> u) = (fun u -> 1*u)`
 - If, inductively, `k` has form `k = fun u -> r*u`,
then the new continuation
`fun v -> k(n*v)`
`= fun v -> (fun u -> r*u) (n*v)`
`= fun v -> r*(n*v)`
`= fun v -> (r*n)*v`
- So integer `r` is a simple way to represent function `k`
- All functions can be made tail-recursive
- Only some continuations can be represented simply

Continuation-passing style in Java

- A Java function written in CPS style

```
interface Cont {  
    int k(int v);  
}
```

To represent
functions $\text{int} \rightarrow \text{int}$

```
static int facc(final int n, final Cont cont) {  
    if (n == 0)  
        return cont.k(1);  
    else  
        return facc(n-1,  
            new Cont() {  
                public int k(int v) {  
                    return cont.k(n * v);  
                }  
            });  
}
```

The new
continuation

Continuation-passing style in Java

- A Java function using anonymous functions

```
class Factorial2 {  
    public static void main(String[] args) {  
        int n = Integer.parseInt(args[0]);  
        System.out.println(facc(n, v -> v));  
    }  
  
    static int facc(final int n,  
                    final Function<Integer,Integer> k) {  
        if (n == 0)  
            return k.apply(1);  
        else  
            return facc(n-1, v -> k.apply(n*v));  
    }  
}
```

Anonymous function

The new
continuation

Continuation-passing style in C#

- A C# function in CPS style

```
class Factorial {  
    public static void Main(String[] args) {  
        int n = Int32.Parse(args[0]);  
        Console.WriteLine(facc(n, v => v));  
    }  
}
```

Anonymous function

```
static int facc(int n, Func<int,int> k) {  
    if (n == 0)  
        return k(1);  
    else  
        return facc(n-1, v => k(n*v));  
}
```

The new
continuation

Why make continuations explicit?

- In normal code, the continuation is implicit:
 - The surrounding expressions
 - The next statement
 - The activation records on the stack
- By making the continuation explicit
 - We can ignore it, thus “avoid returning”
 - We can have two continuations, thus “choose how to return”
- Ignoring continuation = throwing exception
- Choosing a continuation is good for
 - handling exceptions, and
 - producing multiple results from an expression

A simple functional language with exceptions

- Let's add exceptions to our small functional language:

```
type expr =  
  | ...  
  | Raise of exn                                // raise exn  
  | TryWith of expr * exn * expr               // try e1 with exn -> e2
```

- Evaluation of an expression now either gives an integer result, or fails (aborts):

```
type answer =  
  | Result of int  
  | Abort of string
```

```
let rec coEval1 e env (cont : int -> answer) : answer =
```

Interpreter with continuation, part 1

```
let rec coEval1 e env (cont : int -> answer) : answer =  
  match e with  
  | CstI i -> cont i  
  | Var x ->  
    match lookup env x with  
    | Int i -> cont i  
    | _ -> Abort "coEval1 Var"  
  | Prim(ope, e1, e2) ->  
    coEval1 e1 env  
    (fun i1 ->  
      coEval1 e2 env  
      (fun i2 ->  
        match ope with  
        | "*" -> cont (i1 * i2)  
        | "+" -> cont (i1 + i2)  
        | ... ))  
  | Raise (Exn s) -> Abort s
```

Compare
lecture 4,
Fun.fs



Comparing with Fun.fs

```
let rec eval (e : expr) (env : value env) : int =  
  match e with  
  | CstI i -> i  
  | Var x ->  
    match lookup env x with  
    | Int i -> i  
    | _ -> failwith "eval Var"  
  | Prim(ope, e1, e2) ->  
    let i1 = eval e1 env  
    let i2 = eval e2 env  
    match ope with  
    | "*" -> i1 * i2  
    | "+" -> i1 + i2  
    | ...
```

No continuation
but same
structure.

Error reporting
through F# build
in exceptions

Interpreter with continuation, part 2

```
let rec coEval1 e env (cont : int -> answer) : answer =  
  match e with  
  | ...  
  | Call(f, eArg) ->  
    let fClosure = lookup env f  
    match fClosure with  
    | Closure (f, x, fBody, fDeclEnv) ->  
      coEval1 eArg env  
        (fun xVal ->  
          let fBodyEnv =  
            (x, Int xVal) :: (f, fClosure) :: fDeclEnv  
            coEval1 fBody fBodyEnv cont)  
    | ...
```

Examples to provoke Abort:

Ex10: foo used as variable
Ex11: Unknown prim.
Ex12: foo not a function

```
let ex10 = Letfun("foo", "y", CstI 3, Var "foo")  
let ex11 = Prim("foo", CstI 3, CstI 4)  
let ex12 = Let("foo", CstI 3, Call("foo", CstI 4))
```



Interpretation of exception *handling*

- Add an error continuation to interpreter:
econt : exn -> answer
- To throw exception, call error continuation instead of normal continuation
- The error continuation looks at the exception and decides whether it wants to handle it
- The error continuation will contain the chain of exceptions to match, i.e., one new lambda added for each TryWith.

Interpreter with two continuations for throwing and handling (part 1)

```
let rec coEval2 e env (cont : int -> answer)
      (econt : exn -> answer) : answer =
  match e with
  | CstI i -> cont i
  | If(e1, e2, e3) ->
    coEval2 e1 env (fun b ->
      if b<>0 then
        coEval2 e2 env cont econ
      else
        coEval2 e3 env cont econ)
    econ
  | ...
  | Raise exn -> econ exn
  | TryWith (e1, exn, e2) ->
    let econ1 thrown =
      if thrown = exn then coEval2 e2 env cont econ
      else econ thrown
    in coEval2 e1 env cont econ1
```



Interpreter with two continuations for throwing and handling (part 2)

The top-level error continuation returns the continuation, adding the text "Uncaught exception"

```
let eval2 e env =  
  coEval2 e env  
    (fun v -> Result v)  
    (fun (Exn s) -> Abort ("Uncaught exception: " + s))
```

Examples:

```
let ex4 =  
  TryWith(Prim("*", CstI 11, Raise (Exn "Outahere")),  
    Exn "Outahere", CstI 999);  
> eval2 ex4 [];;  
val it : answer = Result 999
```

Catch an
exception (ex4)

```
let ex13 = Raise (Exn "Uncaught")  
> eval2 ex13 [];;  
val it : answer = Abort "Uncaught exception: Uncaught"
```

Uncaught
exception (ex13)



Tail position and CPS

- Expressions in tail position are exactly those that are interpreted with the same continuation as the enclosing expression.

```
let rec coEval1 (e : expr) (env : value env)
                (cont : int -> answer) : answer =
match e with
| ...
| Let(x, eRhs, letBody) ->
    coEval1 eRhs env (fun xVal ->
                        let bodyEnv = (x, Int xVal) :: env
                        coEval1 letBody bodyEnv cont)
| ...
```

The `letBody` part is in tail position.

Continuation Based Imperative Interpreter

- We can also model an interpreter for an imperative language with continuations.
- Similar to the functional case, an interpreter that implements throw and try-catch must take a normal continuation and an error continuation.
- The error continuation takes two arguments:
 - the exception and
 - the store at the time where the exception is thrown.

Interpreter for imperative language with two continuations

```
let rec coExec2 stmt (store : naivestore)
  (cont : naivestore -> answer)
  (econt : exn * naivestore -> answer) : answer =
match stmt with
| Asgn(x, e) ->
  cont (setSto store (x, eval e store))
| If(e1, stmt1, stmt2) ->
  if eval e1 store <> 0 then
    coExec2 stmt1 store cont econt
  else
    coExec2 stmt2 store cont econt
| Throw exn ->
  econt(exn, store)
| TryCatch(stmt1, exn, stmt2) ->
  let econt1 (exn1, sto1) =
    if exn1 = exn then coExec2 stmt2 sto1 cont econt
    else econt (exn1, sto1)
  coExec2 stmt1 store cont econt1
| ...
```

```
type answer =
  | Terminate
  | Abort of string
```

```
type exn =
  | Exn of string
```

Continuations and Backtracking

Backtracking:

When a subexpression produces result v that later turns out to be inadequate, the computation may backtrack to that subexpression to ask for a new result v' that may be more adequate.

Expressions in Icon:

The *result sequence* of an expression is the sequence of results it may produce.

The sequence is empty if the expression fails.

Expressions that give multiple results; the Icon language

Expression	Result seq.	Print	Comment
5	5		Constant
write 5	5	5	Constant, side effect
(1 to 3)	1 2 3		Range, 3 results
write (1 to 3)	1 2 3	1	Side effect
every (write (1 to 3))	0	1 2 3	Force all results
(1 to 0)			Empty range, no res.
&fail			No results
(1 to 3)+(4 to 6)	5 6 7 6 7 8 7 8 9		All combinations
3 < 4	4		Comparison success
4 < 3			Comparison fails
3 < (1 to 5)	4 5		Success twice
(1 to 3) (4 to 6)	1 2 3 4 5 6		Each left, each right
(1 to 3) & (4 to 6)	4 5 6 4 5 6 4 5 6		Each right for each left
(1 to 3) ; (4 to 6)	4 5 6		No backtracking to left

Micro-Icon interpreter

- The interpreter takes two continuations:
 - A *failure continuation*
`econt : unit -> value`
called when there are no (more) results
 - A *success continuation*
`cont : value -> econ`
called when there is one (more) result
- The `econt` argument to `cont` can be called by `cont` to ask for more results

```
type value =  
| Int of int  
| Str of string
```

```
let rec eval (e : expr) (cont : cont) (econt : econ) =  
  match e with  
  | CstI i -> cont (Int i) econt  
  | ...  
  | Fail -> econt ()
```

Micro-Icon interpreter, part 1

```
let rec eval (e : expr) (cont : cont) (econt : econ) =  
  match e with  
  | CstI i -> cont (Int i) econ  
  | CstS s -> cont (Str s) econ  
  | Prim(ope, e1, e2) ->  
    eval e1 (fun v1 -> fun econ1 ->  
      eval e2 (fun v2 -> fun econ2 ->  
        match (ope, v1, v2) with  
        | ("+", Int i1, Int i2) ->  
          cont (Int(i1+i2)) econ2  
        | ("*", Int i1, Int i2) ->  
          cont (Int(i1*i2)) econ2  
        | ("<", Int i1, Int i2) ->  
          if i1<i2 then  
            cont (Int i2) econ2  
          else  
            econ2 ()  
        | _ -> Str "unknown prim2")  
      econ1)  
    econ
```

...

5

2+2

Backtrack to
get next
result.



Micro-Icon interpreter, part 2

```
let rec eval (e : expr) (cont : cont) (econt : econ) =  
  match e with  
  | ...  
  | FromTo(i1, i2) ->  
    let rec loop i =  
      if i <= i2 then  
        cont (Int i) (fun () -> loop (i+1))  
      else  
        econ ()  
    in loop i1  
  | Write e ->  
    eval e (fun v ->  
      fun econ1 -> (write v; cont v econ1))  
    econ  
  | If(e1, e2, e3) ->  
    eval e1 (fun _ -> fun _ -> eval e2 cont econ)  
    (fun () -> eval e3 cont econ)  
  | ...
```

Build econ
for backtrack
next result

(1 to 3)

Sequence is empty -
backtrack further.

write 5

if 3<4 ...

We cannot backtrack to e1 and result
value of e1 is not used.

Micro-Icon interpreter, part 3

```
let rec eval (e : expr) (cont : cont) (econt : econ) =  
  match e with  
  | ...  
  | And(e1, e2) ->  
    eval e1 (fun _ -> fun econ1 -> eval e2 cont econ1) econ  
  | Or(e1, e2) ->  
    eval e1 cont (fun () -> eval e2 cont econ)  
  | Seq(e1, e2) ->  
    eval e1 (fun _ -> fun econ1 -> eval e2 cont econ)  
    (fun () -> eval e2 cont econ)  
  | Every e ->  
    eval e (fun _ -> fun econ1 -> econ1 ())  
    (fun () -> cont (Int 0) econ)
```

Result of e1 is ignored. **e1&e2**

First all of e1 and then all of e2 **e1|e2**

Ignore result of e1 and no backtrack into e1. **e1;e2**

econ1 ignored. **every e**

Take the result, ignore it,
and ask for one more

When no more results,
succeed with result 0



Micro-Icon interpreter, examples

Examples:

```
// (write(1 to 3)) ; fail
let ex1 = Seq(Write (FromTo(1, 3)), Fail);
> run ex1;;
1 Failed
val it : value = Int 0
```

```
// (write(1 to 3)) & fail
let ex2 = And(Write (FromTo(1, 3)), Fail);
> run ex2;;
1 2 3 Failed
val it : value = Int 0
```

```
// (write((1 to 3) & (4 to 6))) & fail
let ex3and = And(Write(And(FromTo(1, 3), FromTo(4, 6))), Fail);
> run ex3and;;
4 5 6 4 5 6 4 5 6 Failed
val it : value = Int 0
```

```
> dotnet fsi Icon.fs
> open Icon;;
```

Sequence ";" provides no back tracking to the left. Therefore, writes one and then fails.

And "&" do each right for each left, hence fails and tries next until all tried.

Notice & inside write leading to 4 5 6 being written 3 times.



Micro-Icon interpreter, examples

Examples:

```
> dotnet fsi Icon.fs  
> open Icon;;
```

```
// (write((1 to 3) | (4 to 6))) & fail  
let ex3or = And(Write(Or(FromTo(1, 3), FromTo(4, 6))), Fail);  
> run ex3or;;  
1 2 3 4 5 6 Failed  
val it : value = Int 0
```

Or "|" do each left and right inside write.

```
// write((1 to 3) & ((4 to 6) & "found"))  
let ex4 = Write(And(FromTo(1, 3), And(FromTo(4, 6), CstS "found")));  
> run ex4;;  
found  
val it : value = Str "found"
```

And "&" do each right for each left, which leads to "found".

```
// every(write(4 < (1 to 10)))  
let ex9 = Every(Write(Prim("<", CstI 4, FromTo(1, 10))));  
> run ex9;;  
5 6 7 8 9 10  
val it : value = Int 0
```

4 < X succeeds for 5 to 10
every force all solutions.

